



What are we going to see in this session?

- Understanding Terraform HCL
- Arguments & Blocks
- Providers
- Resource
- Provisioner
- Output
- Terraform state
- Terraform work flow
- Introduction to variables
- Terraform variable types
- Terraform set
- How to call declared variables
- Most commonly used variable types



Understanding Terraform HCL

- Terraform codes should be written in HCL [HashiCorp Configuration Language]
- Let's try to understand the terraform syntax and rules.
- Terraform can only understand the files which are with `[tf]` extension.

example: `main.tf`



Argument & Blocks

- Blocks : Blocks is a container for other contents.

```
block {  
    argument_1 = "value"  
    argument_2 = "value"  
}
```

- Arguments : An argument assigns a value to a particular name.

```
argument = "abc123"
```

- The identifier before [=] sign is argument name & the expression after [=] sign is argument value.
- The argument values can be
 - ✓ String
 - ✓ Number
 - ✓ Bool
 - ✓ List
 - ✓ Map



Providers

- Providers are responsible for understanding and take care of API interactions and expose required resources.
- As mentioned earlier TF supports many providers some samples are
- AWS

```
provider "aws" {  
}
```

- Azure

```
provider "azurerm" {  
}
```

- VMware

```
provider "vsphere" {  
}
```



Resources

After establishing the connection to providers what are we going to do? “We are going to utilize there resources”

In modern cloud world, someone will have these resource ready and you can utilize it on demand.

“ Hence resources means infrastructure components”

```
Resource "resource_type" "resource_name" {  
}
```



Provisioner

Using Provisioner you can do changes in instances.

Common uses cases are :

Copying some files to remote servers.

Executing script on newly created instances.

Inject bootstrapping to instances.

Clean up some thing before machine is destroyed



Output

Shows values that are highlighted after terraform applies.
This output is also maintained in terraform state file.



Terraform State

Terraform state stores status of your managed infrastructure & configuration.

This state is used by Terraform to map real world resources to your configuration, keep track of meta data and to improve performance of large scale infrastructure.

Terraform state is generally stored in a file locally named as “terraform.tfstate” but it can be also stored elsewhere which can work better in team environment..



Terraform work flow

Write : Author your code of how your infrastructure should be.
Plan : Preview changes before apply
Apply : Provision your infrastructure.



End of this topic !

Any questions?

TERRAFORM